PipeWatch

CI/CD Pipeline Status Notifier in AWS

Team Name: ERROR 420

Team Leader: Athish M
Register Number: 7376241CS136
Team Member: Vijay Haripriyan D
Register Number: 7376242AD341

1. Problem Statement

1.1 Original Problem Statement

Problem Description:
Modern software development teams rely on Continuous Integration and Continuous Deployment (CI/CD) pipelines to automate building, testing, and deployment. Organizations using AWS CodePipeline, CodeBuild, CodeDeploy, Lambda, and CloudWatch often struggle to identify failed builds, deployment errors, and infrastructure issues quickly. Develop a cloud-based solution that continuously monitors AWS CI/CD pipeline events, detects failures in real time, categorizes incidents, and automatically sends notifications through Email, Slack, Microsoft Teams, or SMS. The platform should provide deployment history, failure analysis, pipeline health dashboards, and predictive insights to reduce downtime and improve DevOps efficiency.

1.2 Existing System

In most existing setups, pipeline monitoring is scattered across multiple standalone tools. Jenkins provides its own build console, AWS provides its own service dashboards, and error logs are usually inspected manually through SSH or CloudWatch. There is no single place where a developer can see the pipeline status, the deployment history, and the health of the underlying AWS infrastructure together.

1.3 Problems Faced in the Existing System

- No centralized view of CI/CD pipeline status and AWS infrastructure health.
- Manual switching between Jenkins console, AWS Console, and terminal logs.
- Delayed detection of build failures and deployment issues.
- Difficult to track deployment history and correlate it with infrastructure events.
- No real-time visibility into build queue, executors, or current pipeline stage.
- Lack of automated notifications when a pipeline event occurs.

1.4 Proposed Solution

To address these problems, we propose PipeWatch, a centralized DevOps monitoring platform that integrates Jenkins CI/CD pipelines with AWS cloud services and displays everything through a single, real-time React-based dashboard. The system collects pipeline execution data, deployment status, build history, and AWS infrastructure information through backend APIs, and presents it through interactive charts, KPI cards, and status panels so that a DevOps engineer can monitor the complete deployment lifecycle from one screen.

1.5 Objectives

- Build a centralized dashboard for monitoring CI/CD pipeline execution and AWS infrastructure.
- Provide real-time visibility into build status, current pipeline stage, and deployment history.
- Automate event capture using AWS EventBridge, Lambda, and DynamoDB.
- Reduce the time required to detect and respond to pipeline or deployment failures.
- Provide automated notifications for key pipeline events using n8n.

1.6 Scope of the Project

The scope of this project covers the design and development of the PipeWatch monitoring platform, including the Jenkins pipeline configuration, AWS service integration (IAM, EC2, S3, Lambda, EventBridge, API Gateway, DynamoDB), the Node.js/Express backend, and the React.js frontend dashboard. The project focuses on monitoring and visibility rather than replacing Jenkins or AWS themselves; it acts as a unified observability layer on top of the existing CI/CD and cloud infrastructure.

2. Project Overview

2.1 Introduction

PipeWatch is a cloud-based DevOps monitoring platform built to give development teams a single, unified view of their CI/CD pipelines and AWS infrastructure. It brings together Jenkins build data, AWS service health, and deployment history into one interactive dashboard, removing the need to constantly switch between tools during a deployment.

2.2 Why This Project Was Selected

As a team working with AWS and DevOps tools, we repeatedly experienced the difficulty of tracking a deployment across multiple dashboards — Jenkins for build logs, the AWS Console for infrastructure status, and separate tools for notifications. This project was selected to solve a problem we personally encountered, and because it combines core cloud computing and DevOps concepts — CI/CD automation, serverless processing, event-driven architecture, and full-stack development — into a single practical system.

2.3 Motivation

The motivation behind PipeWatch comes from the growing complexity of modern deployment pipelines. As teams adopt more cloud services, the number of moving parts that can fail also increases. A centralized monitoring dashboard reduces the cognitive load on developers, speeds up failure detection, and improves confidence in the deployment process — all of which directly improve team productivity.

2.4 Expected Outcome

The expected outcome is a fully functional monitoring dashboard that displays live Jenkins pipeline status, deployment history, and AWS infrastructure health, backed by an automated event pipeline using EventBridge, Lambda, and DynamoDB, with optional notifications through n8n. The system should reduce the time taken to identify a failed build or deployment issue and give the team clear analytics on deployment frequency and success rate.

2.5 Features

- Live Jenkins pipeline monitoring with current build status and stage.
- Deployment history with commit details, environment, and timestamps.
- Real-time AWS infrastructure health monitoring (EC2, S3, Lambda, API Gateway, EventBridge, DynamoDB).
- Interactive charts for build duration, success/failure rate, and deployment frequency.
- Auto-refreshing dashboard with no manual page reload required.
- Event-driven backend using AWS EventBridge and Lambda for near real-time updates.
- Automated notifications for pipeline events through n8n workflows.

3. Technologies Used

This section lists every technology used in PipeWatch along with the purpose it serves, why it was chosen over alternatives, and how it was applied in the project.

3.1 Frontend

React.js

Purpose: Build the interactive, component-based dashboard UI.

Why Selected: Large ecosystem, reusable components, and strong support for real-time data-driven UIs.

How It Was Used: Used to build the Pipeline Monitor, Deployment History, Infrastructure, and Charts pages.

Vite

Purpose: Development server and build tool for the React application.

Why Selected: Much faster cold-start and hot-reload compared to traditional bundlers like Webpack.

How It Was Used: Used to run the dashboard in development and to produce the optimized production build deployed to S3.

Recharts

Purpose: Render interactive charts and graphs.

Why Selected: Native React component API that integrates cleanly with component state and props.

How It Was Used: Used to plot build duration trend, success/failure distribution, and deployment frequency.

CSS

Purpose: Style and layout the dashboard.

Why Selected: Full control over a lightweight, custom design without pulling in a heavy UI framework.

How It Was Used: Used for the dashboard layout, KPI cards, status colours, and responsive grid.

3.2 Backend

Node.js

Purpose: JavaScript runtime for the backend server.

Why Selected: Non-blocking, event-driven model well suited to polling Jenkins and AWS APIs concurrently.

How It Was Used: Runs the Express server that powers all REST APIs consumed by the dashboard.

Express.js

Purpose: Web framework for building REST APIs.

Why Selected: Minimal, unopinionated, and quick to set up routes and middleware.

How It Was Used: Used to expose endpoints for pipeline status, build history, and infrastructure data.

3.3 DevOps

GitHub

Purpose: Source code hosting and version control.

Why Selected: Industry-standard integrates natively with Jenkins via webhooks.

How It Was Used: Hosts the project repository and triggers the Jenkins pipeline on every push.

Jenkins

Purpose: CI/CD automation server.

Why Selected: Highly configurable Declarative Pipelines and wide plugin support.

How It Was Used: Runs the build, validation, and deployment pipeline for the static site to S3.

n8n

Purpose: Workflow automation and notifications.

Why Selected: Open-source, self-hostable, and easy to connect to EventBridge/Lambda outputs without writing custom notification code.

How It Was Used: Used to send automated notifications when a deployment succeeds or fails.

3.4 AWS Services

IAM

Purpose: Identity and access management.

Why Selected: Required to securely control which users and services can access which AWS resources.

How It Was Used: Used to create a restricted IAM user and access keys for CLI and pipeline access instead of using root credentials.

EC2

Purpose: Virtual server to host Jenkins.

Why Selected: Full control over the operating system needed to install and configure Jenkins.

How It Was Used: Hosts the Jenkins server that runs the CI/CD pipeline.

S3

Purpose: Static website hosting and deployment target.

Why Selected: Low-cost, highly available storage with native static website hosting support.

How It Was Used: Jenkins deploys the built website to an S3 bucket configured for static hosting.

Lambda

Purpose: Serverless event processing.

Why Selected: No server management, scales automatically, and runs only when triggered.

How It Was Used: Processes deployment events published by Jenkins/EventBridge and writes structured records to DynamoDB.

EventBridge

Purpose: Event bus for routing pipeline events.

Why Selected: Decouples the event producer (Jenkins) from the event consumer (Lambda), making the system easier to extend.

How It Was Used: Receives deployment events and routes them to the correct Lambda function.

API Gateway

Purpose: Expose backend/Lambda functionality as REST endpoints.

Why Selected: Managed, scalable API layer that integrates directly with Lambda.

How It Was Used: Provides REST endpoints used by the backend and dashboard to fetch processed event data.

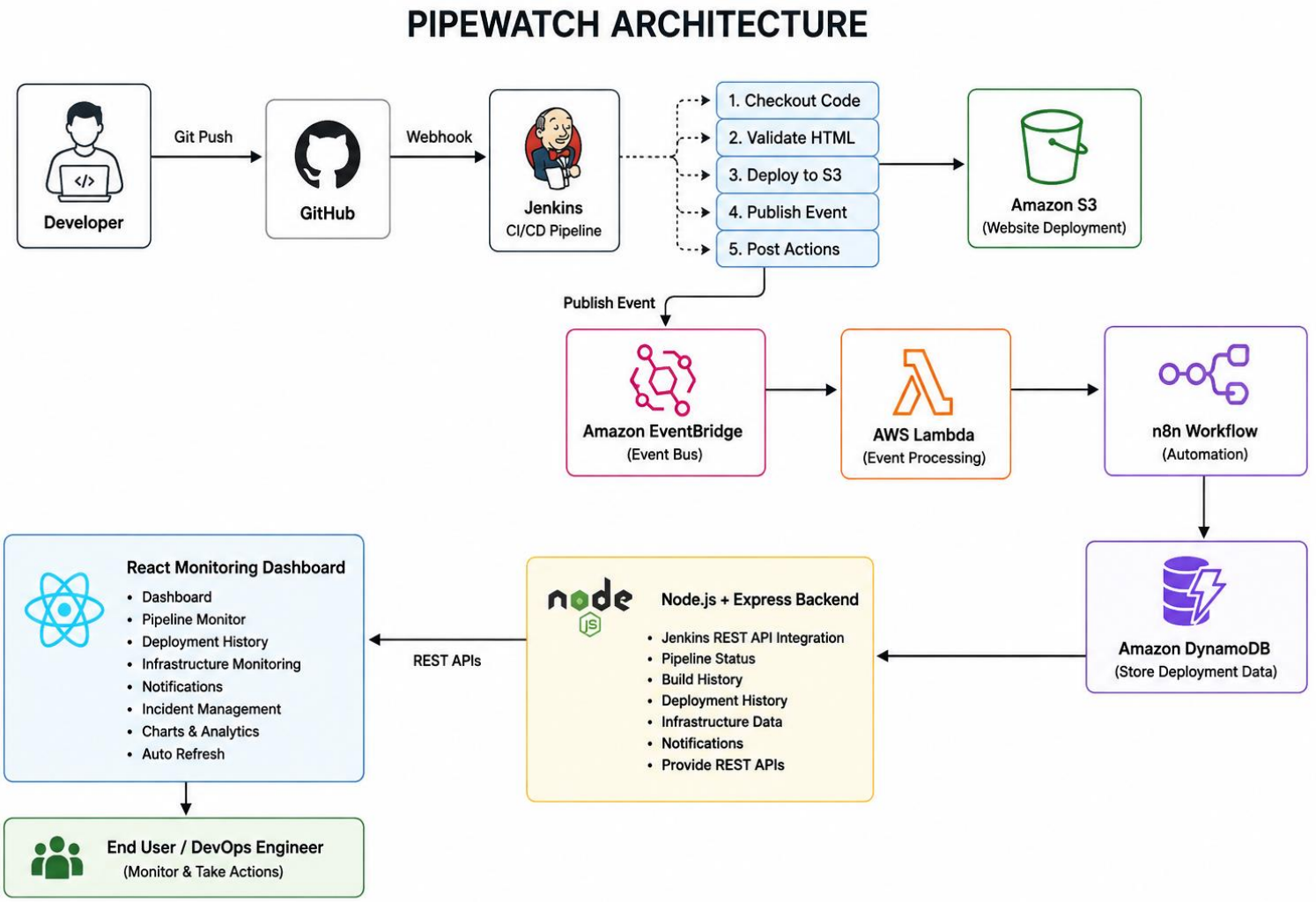
DynamoDB

Purpose: Store deployment and pipeline event records.

Why Selected: Serverless NoSQL database with low-latency reads, ideal for storing frequently updated event data.

How It Was Used: Stores deployment history and infrastructure event records used by the dashboard.

Pipe Monitoring Architecture:



4. AWS Cloud Implementation

This section explains, service by service, how each AWS component was configured and integrated into PipeWatch.

4.1 IAM

- Created a dedicated IAM user for the project instead of using the AWS root account.
- Attached least-privilege policies covering only EC2, S3, Lambda, EventBridge, API Gateway, and DynamoDB access.
- Generated Access Keys for the IAM user for programmatic access.
- Configured the AWS CLI locally and on the EC2 instance using 'aws configure' with the generated access key, secret key, region, and output format.

4.2 EC2

- Launched an EC2 instance to host the Jenkins server.
- Configured a Security Group to allow inbound traffic on port 22 (SSH) and port 8080 (Jenkins) from the required IP ranges.
- Connected to the instance using SSH with a key pair.
- Installed Java and Jenkins on the instance and started the Jenkins service.

4.3 S3

- Created an S3 bucket and enabled static website hosting.
- Configured a bucket policy to allow public read access to the hosted website files.
- The Jenkins pipeline's deploy stage uploads the built website files to this bucket using the AWS CLI ('aws s3 sync').
- Every successful pipeline run updates the live site automatically through this bucket.

4.4 EventBridge

- Created a custom EventBridge event bus to receive deployment events from the pipeline.
- Defined event rules to route incoming deployment events to the correct target based on event type.
- Configured the rule target as the processing Lambda function.

4.5 Lambda

- Created a Lambda function to process incoming deployment events.
- Configured EventBridge as the trigger for the function.
- The function parses the event payload, extracts build/deployment details, and writes a structured record to DynamoDB.

4.6 DynamoDB

- Created a DynamoDB table to store deployment and pipeline event records.
- Defined a partition key (build/event ID) for fast lookups.
- Every event processed by Lambda is stored here, forming the deployment history used by the dashboard.

4.7 API Gateway

- Created a REST API in API Gateway to expose deployment data stored in DynamoDB.
- Configured routes and integrated them with the backend/Lambda for read access.
- The Node.js backend and/or the React dashboard call these endpoints to retrieve processed event data.

4.8 n8n

- Created an n8n workflow that listens for new deployment events.
- Configured automation steps to format the event data into a readable message.
- Connected a notification node so the team is automatically alerted when a deployment succeeds or fails.

Project Demonstration:

This project was developed entirely on **Amazon Web Services (AWS)**, with all major components—including the Jenkins server, n8n server, Amazon EC2, AWS Lambda, Amazon API Gateway, Amazon DynamoDB, Amazon EventBridge, Amazon S3, IAM, CloudWatch, and other project-specific cloud resources—successfully deployed, integrated, tested, and demonstrated during the project development phase.

To optimize AWS Academy credits, prevent unnecessary cloud costs, and follow responsible cloud resource management practices, **all AWS resources created specifically for this project have been intentionally decommissioned and deleted** after the successful completion and demonstration of the project.

Additionally, the **AWS Academy Cloud account allocated for this project is scheduled to expire on 05/08/2026**. As a result, the live cloud infrastructure and deployed application will no longer be accessible after this date.

To ensure the complete implementation and functionality of the project can still be reviewed, a comprehensive demonstration video has been provided. The video includes:

- Complete project workflow
- AWS cloud architecture
- Jenkins CI/CD pipeline execution
- GitHub integration
- AWS service integration
- Event-driven workflow
- Real-time monitoring dashboard
- Deployment process
- Notification workflow
- Final project output

Kindly refer to the demonstration video below to view the complete working implementation of the project before the AWS resources were decommissioned and the AWS Academy Cloud account expired.

Project Demonstration Video:

Video Link:
https://drive.google.com/file/d/1PxiX3IRlv7XKR_xTLVkxjkRzL_O7megi/view?usp=sharing

5. Jenkins CI/CD Implementation

5.1 Step-by-Step Setup

1. Installed Jenkins on the EC2 instance and completed the initial admin setup.
2. Installed required plugins, including the GitHub Integration and Pipeline plugins.
3. Connected Jenkins to the project's GitHub repository using a personal access token / SSH credentials.
4. Configured a GitHub Webhook so that every push to the repository automatically triggers the Jenkins pipeline.
5. Created a Jenkins Declarative Pipeline (Jenkinsfile) defining the build stages.
6. Ran the pipeline and verified successful build execution and deployment.

5.2 Pipeline Stages

Checkout — pulls the latest source code from the GitHub repository

↓

Validate HTML — checks the website files for structural correctness before deployment

↓

Deploy to S3 — uploads the validated build to the S3 static hosting bucket

↓

Publish Event — sends a deployment event to EventBridge for downstream processing

↓

Post Actions — cleans up the workspace and reports the final build status

6. Backend Development

The backend is built with Node.js and Express.js, and is responsible for collecting, processing, and exposing all real-time DevOps data consumed by the dashboard.

6.1 Folder Structure

```
backend/
├── src/
│   ├── routes/          → API route definitions
│   ├── controllers/     → request handling logic
│   ├── services/        → Jenkins & AWS integration logic
│   ├── middleware/      → auth, error handling, logging
│   └── utils/           → helper functions
├── app.js               → Express app setup
└── server.js            → server entry point
```

6.2 REST APIs

- GET /api/pipeline/status — returns the current pipeline status and stage.
- GET /api/pipeline/history — returns the build/deployment history.
- GET /api/infrastructure — returns AWS infrastructure health data.
- GET /api/metrics — returns aggregated data used to render charts.

6.3 Jenkins REST API Integration

The backend connects to Jenkins using its built-in REST API to fetch the latest build number, build status, current stage, queue length, and executor status. This data is polled at a regular interval and cached briefly to avoid overloading the Jenkins server.

6.4 Data Processing

Raw responses from Jenkins and AWS are transformed into a consistent JSON structure before being sent to the frontend — for example, converting Jenkins' raw timestamps and build results into a simplified status field (success, failure, or in-progress) that the dashboard can render directly.

6.5 Error Handling

Centralized error-handling middleware catches failures from Jenkins/AWS API calls (timeouts, authentication errors, unreachable services) and returns a clean, consistent error response instead of letting the request crash silently.

6.6 Auto Refresh

The frontend polls the backend APIs at a fixed interval, and the backend is designed to respond quickly using lightweight caching so the dashboard can auto-refresh every few seconds without noticeable lag.

6.7 Authentication

API access is protected using a simple token/key-based check on backend routes, and all AWS calls made by the backend use restricted IAM credentials rather than root access.

6.8 API Responses

All API responses follow a consistent JSON envelope containing a status field, a data field, and, where relevant, a message field — making it predictable for the React frontend to consume.

7. Frontend Development

The frontend is a React.js single-page application that renders all monitoring data collected by the backend.

7.1 Folder Structure

```
frontend/
├── src/
│   ├── components/      → reusable UI components
│   ├── pages/           → Dashboard, Pipeline, History, Infra
│   ├── charts/          → Recharts-based chart components
│   ├── services/        → API call functions
│   ├── hooks/           → custom React hooks (auto-refresh, etc.)
│   └── App.jsx          → routing and layout
└── vite.config.js
```

7.2 Components

- KPI Cards — show latest build number, status, success rate, and failure rate at a glance.
- Status Badges — colour-coded indicators for build/deployment status.
- Data Tables — used for deployment history and event logs.

7.3 Routing

Client-side routing separates the application into distinct views — Dashboard, Pipeline Monitor, Deployment History, Infrastructure, and Notifications — so each concern has its own dedicated page while sharing a common layout and navigation bar.

7.4 Dashboard

The main landing page aggregates the most important information into KPI cards and summary charts, giving an at-a-glance view of overall system health.

7.5 Pipeline Monitor

Displays the latest build number, build status, current pipeline stage, build duration, queue length, and executor status, refreshed automatically.

7.6 Deployment History

Lists past deployments with build number, commit details, deployment status, timestamp, and environment, sourced from the DynamoDB-backed history API.

7.7 Infrastructure

Shows the live health of EC2, S3, Lambda, API Gateway, EventBridge, and DynamoDB, pulled through the backend's infrastructure endpoint.

7.8 Notifications

Surfaces recent pipeline and deployment events so the user does not need to check n8n or their email separately while viewing the dashboard.

7.9 Incident Management

Failed builds or deployments are flagged distinctly on the dashboard so they can be tracked and investigated instead of being buried in general history.

7.10 Charts

- Pipeline Trend — build outcomes over time.
- Success Rate — percentage of successful builds.
- Failure Rate — percentage of failed builds.
- Build Duration — time taken per build, plotted over recent builds.
- Deployment Trend — frequency of deployments over time.
- Infrastructure Usage — health/activity of AWS services over time.

8. Architecture

This section presents the system's architecture as a set of simple flow diagrams describing how data and requests move through PipeWatch.

8.1 Overall Architecture

Developer (GitHub Repository)

↓

Jenkins CI/CD Server (EC2)

↓

AWS Services — S3 / EventBridge / Lambda / DynamoDB / API Gateway

↓

Node.js + Express Backend

↓

React.js Dashboard (PipeWatch)

8.2 AWS Architecture

IAM — access control for all AWS resources

↓

EC2 — hosts Jenkins

↓

S3 — static website hosting / deployment target

↓

EventBridge — routes deployment events

↓

Lambda — processes events

↓

DynamoDB — stores event & deployment records

↓

API Gateway — exposes stored data via REST

8.3 CI/CD Flow

Code Push to GitHub

↓

Webhook Triggers Jenkins

↓

Checkout → Validate → Deploy to S3

↓

Publish Deployment Event to EventBridge

↓

Post Build Actions

8.4 Backend Flow

Backend polls Jenkins REST API and AWS services

↓

Data is processed and normalised

↓

Processed data is cached briefly for fast responses

↓

REST APIs serve the processed data to the frontend

8.5 Dashboard Flow

Dashboard requests data from backend REST APIs on load

↓

Auto-refresh hook polls the APIs at fixed intervals

↓

New data updates KPI cards, tables, and charts in place

↓

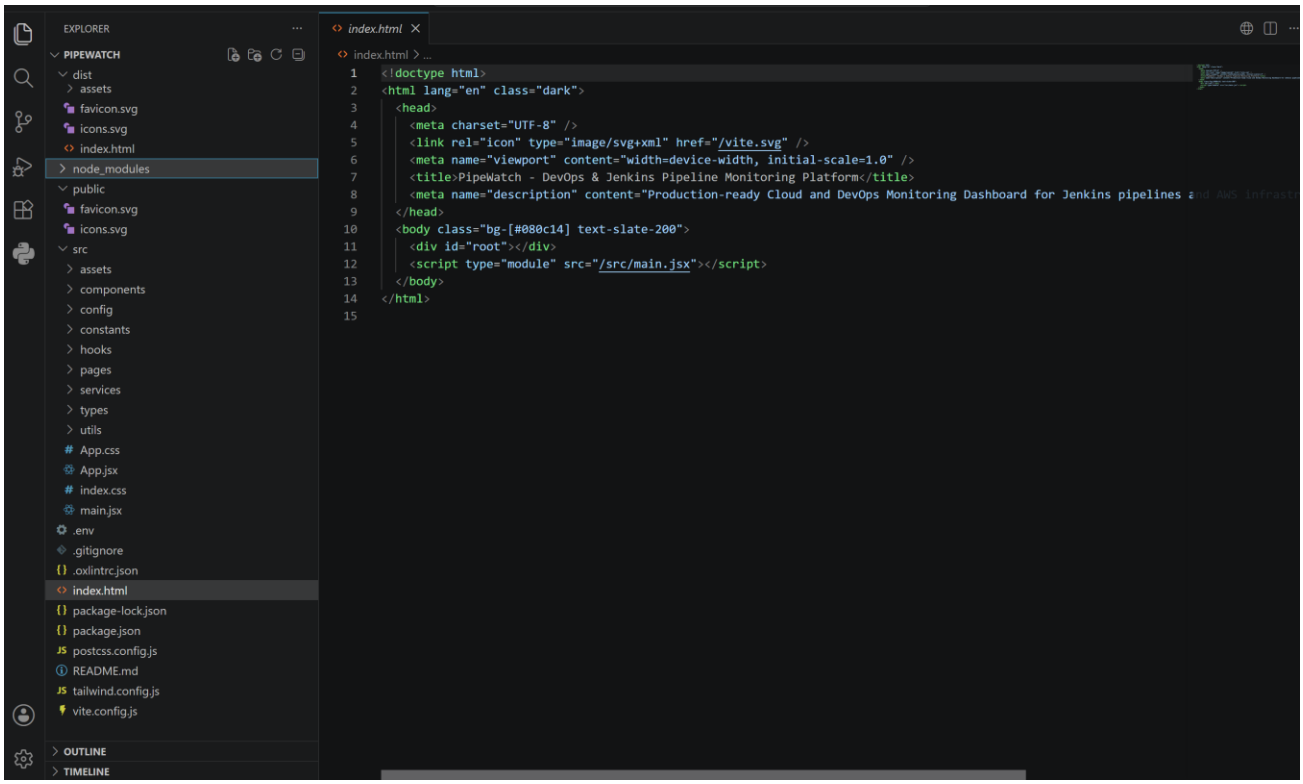
User views live pipeline, deployment, and infrastructure status

10. Results

10.1 Screenshots

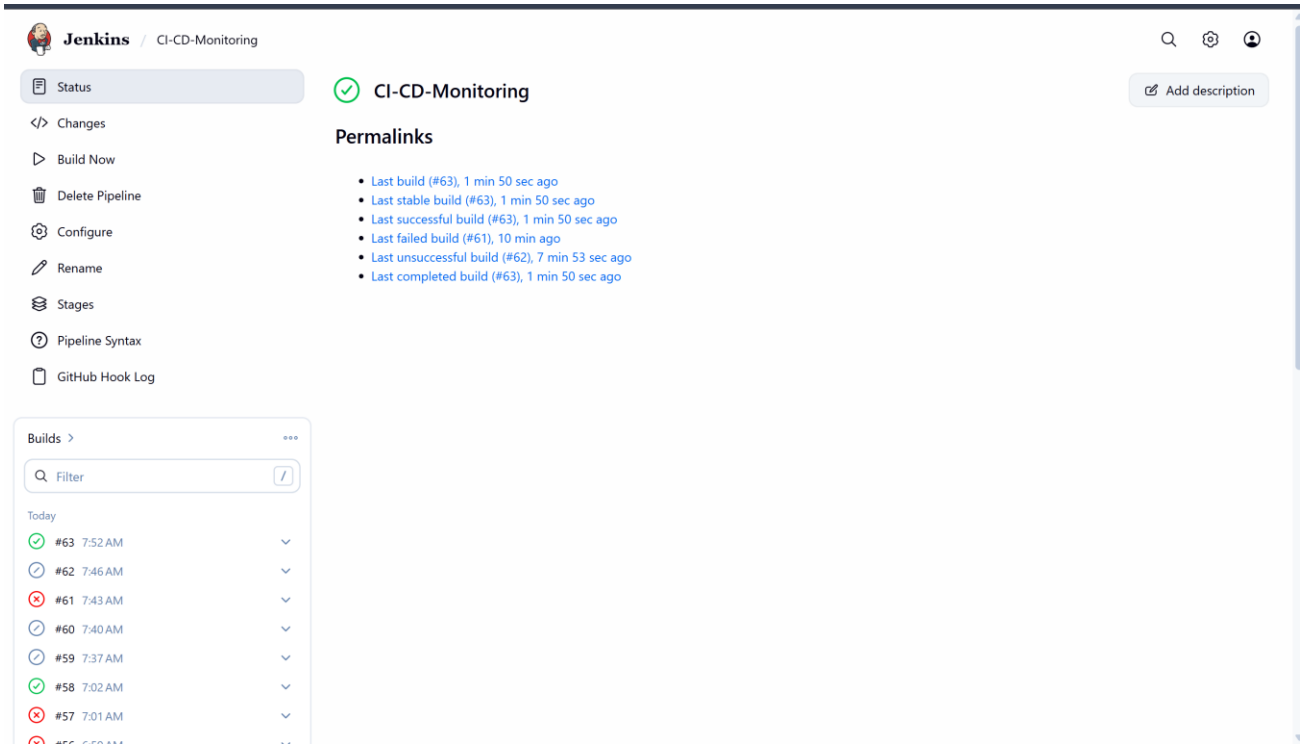
Project Images:

Frontend:



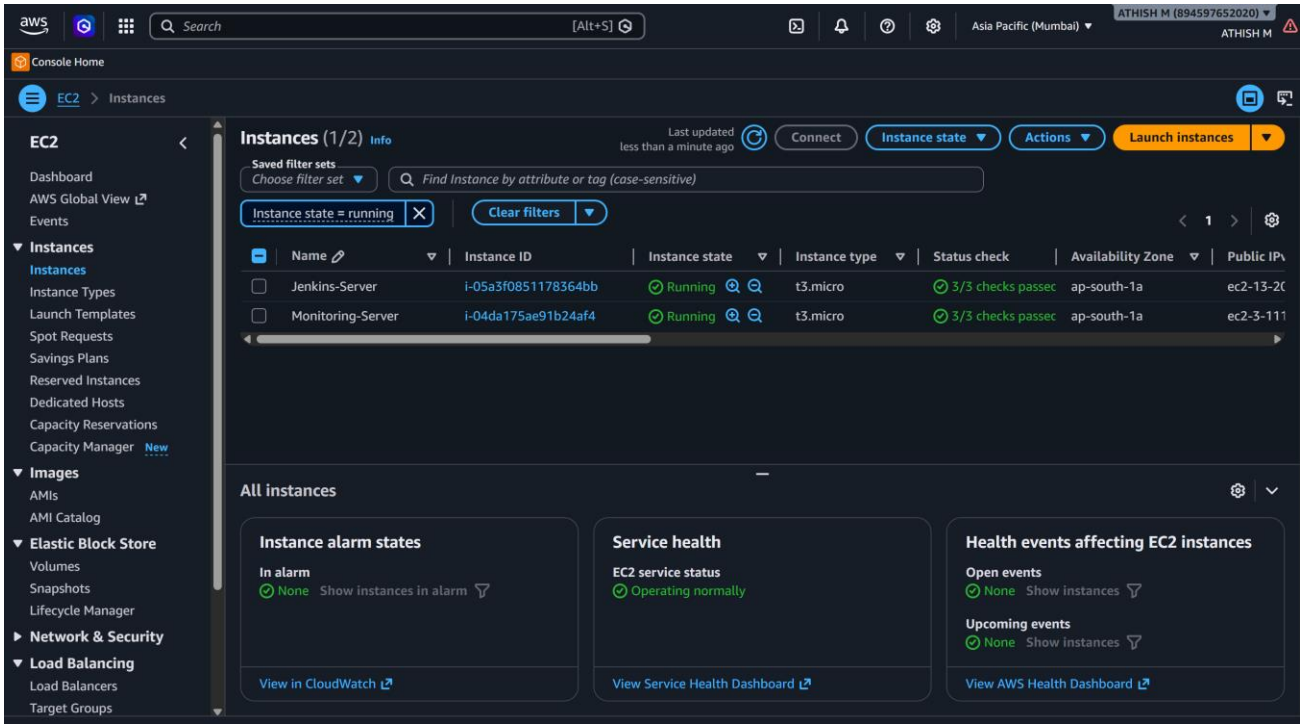
CI/CD:

JENKINS:

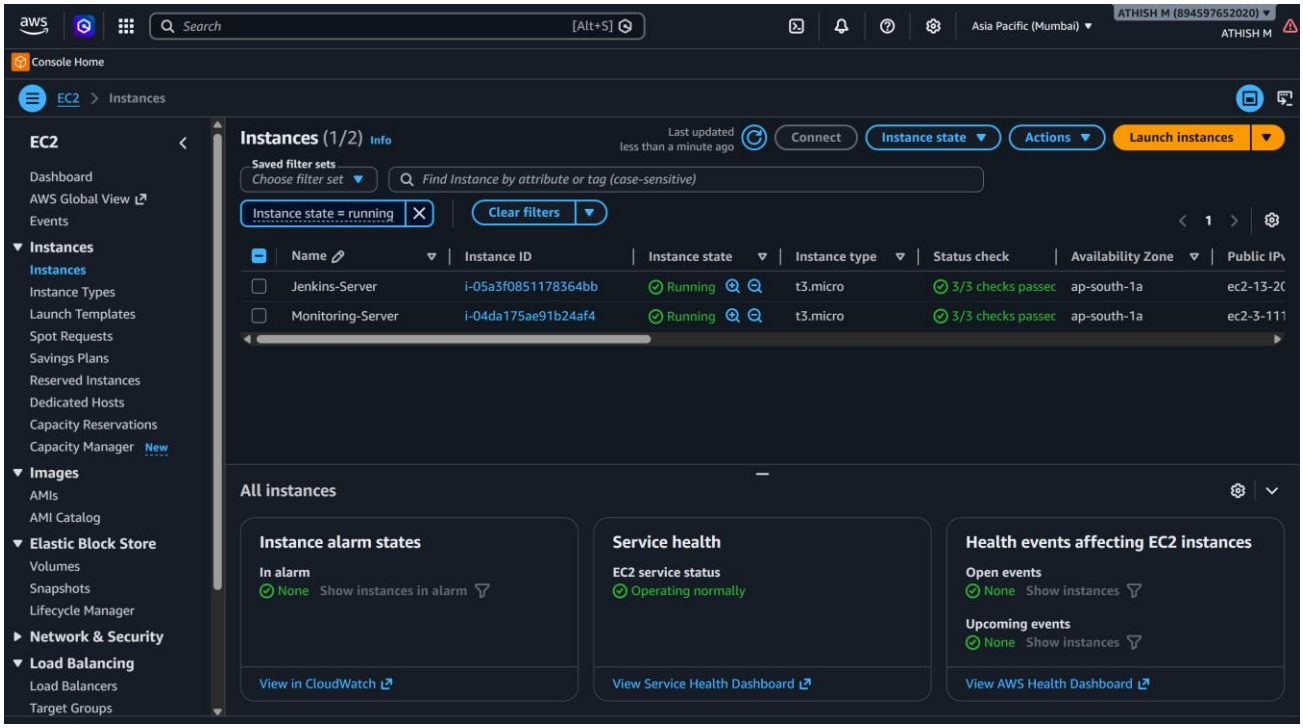


AWS:

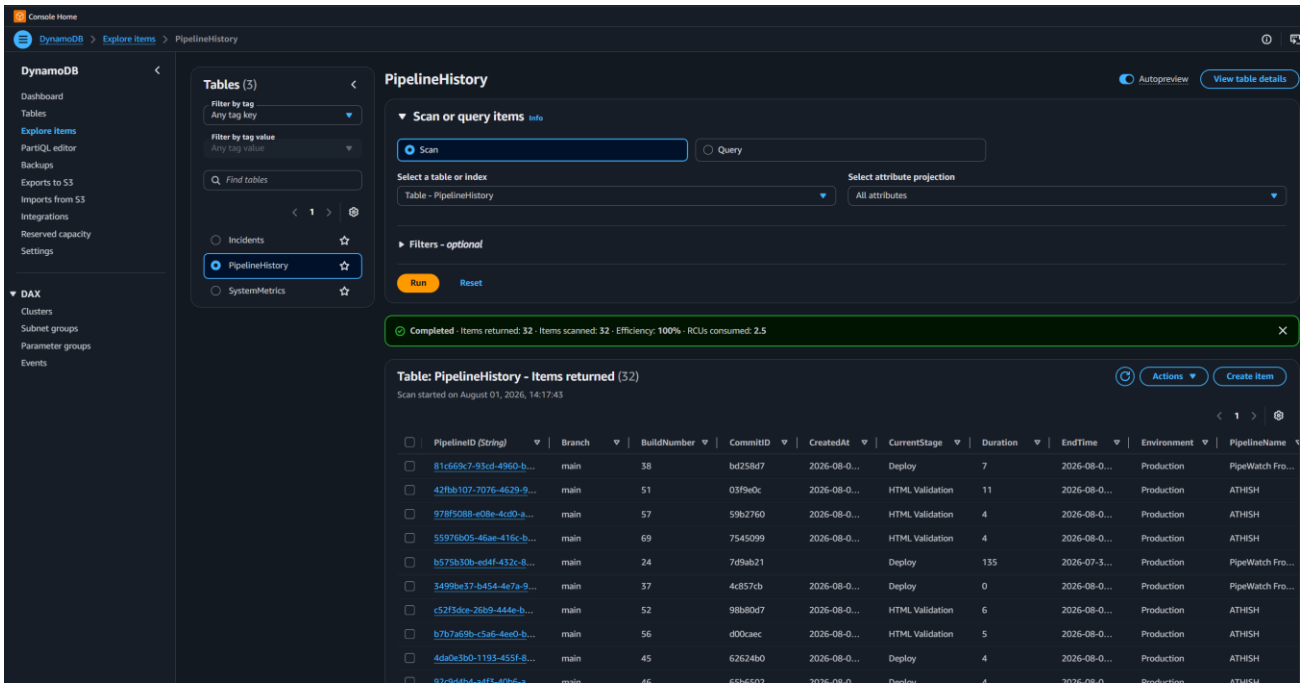
EC2:



Lambda:

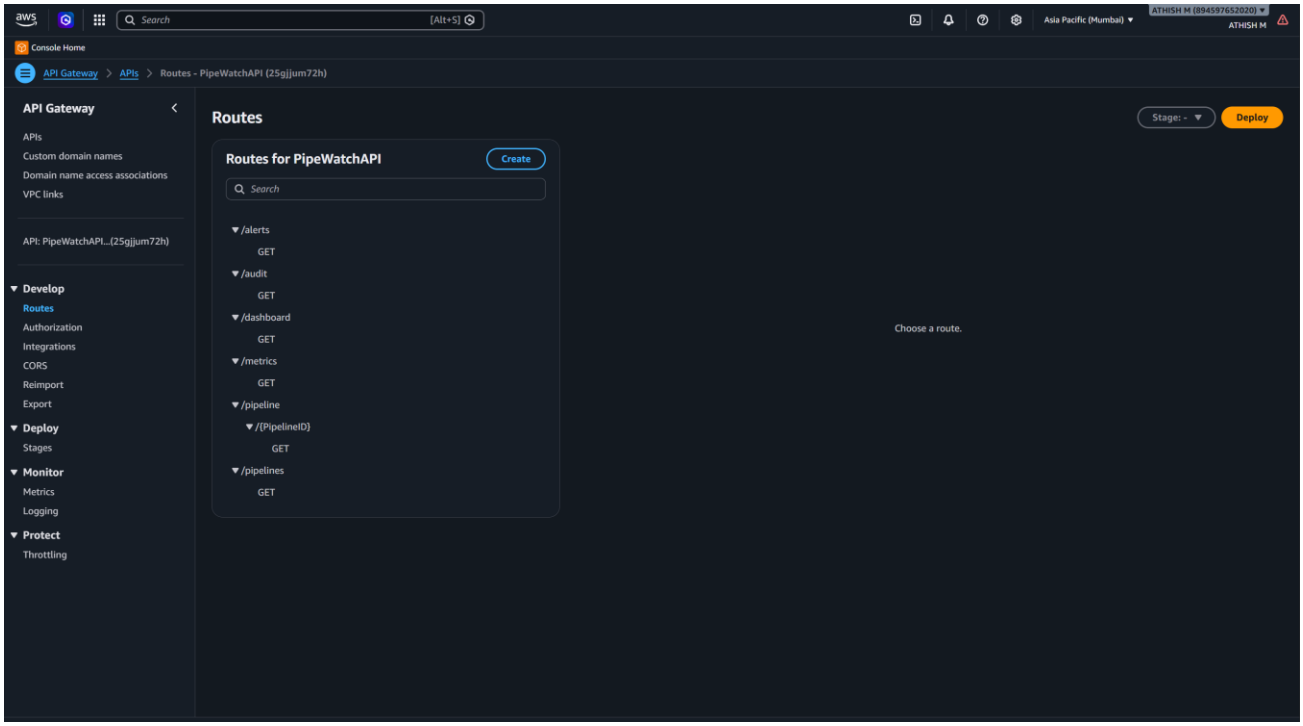


DYNAMO DB:

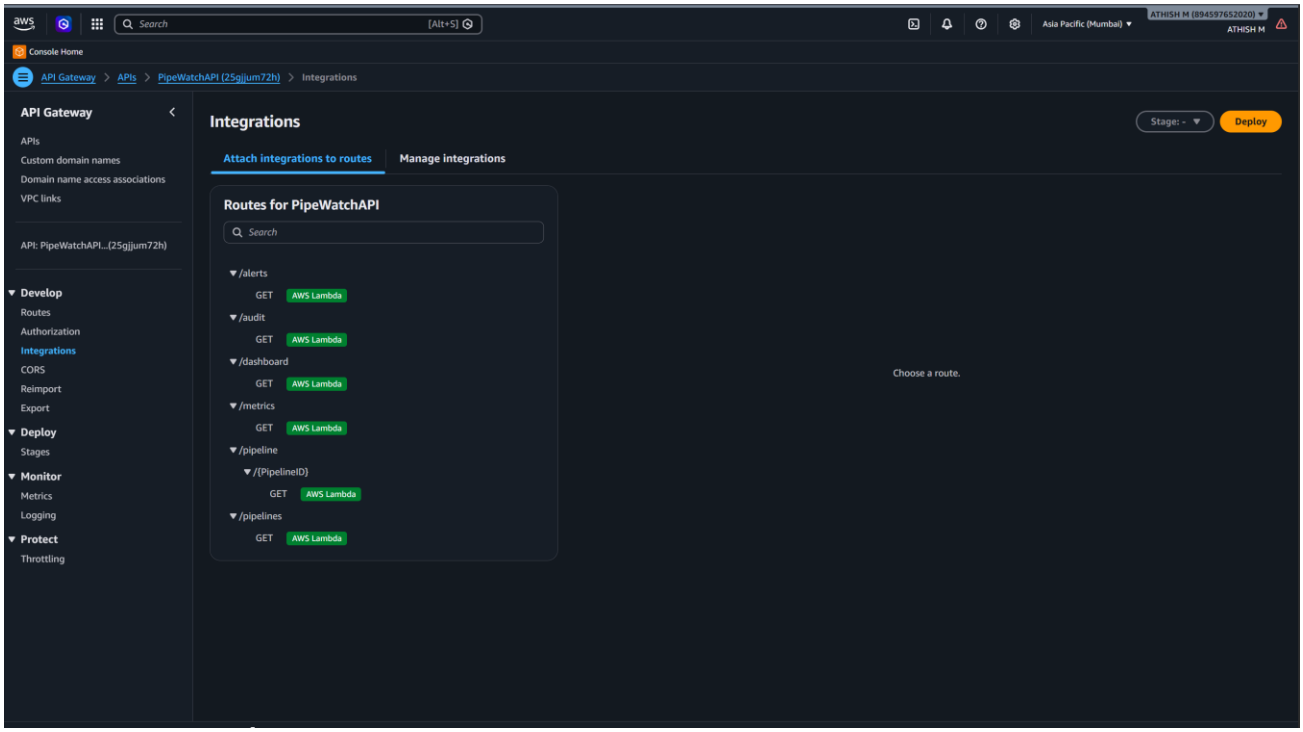


API GATEWAY:

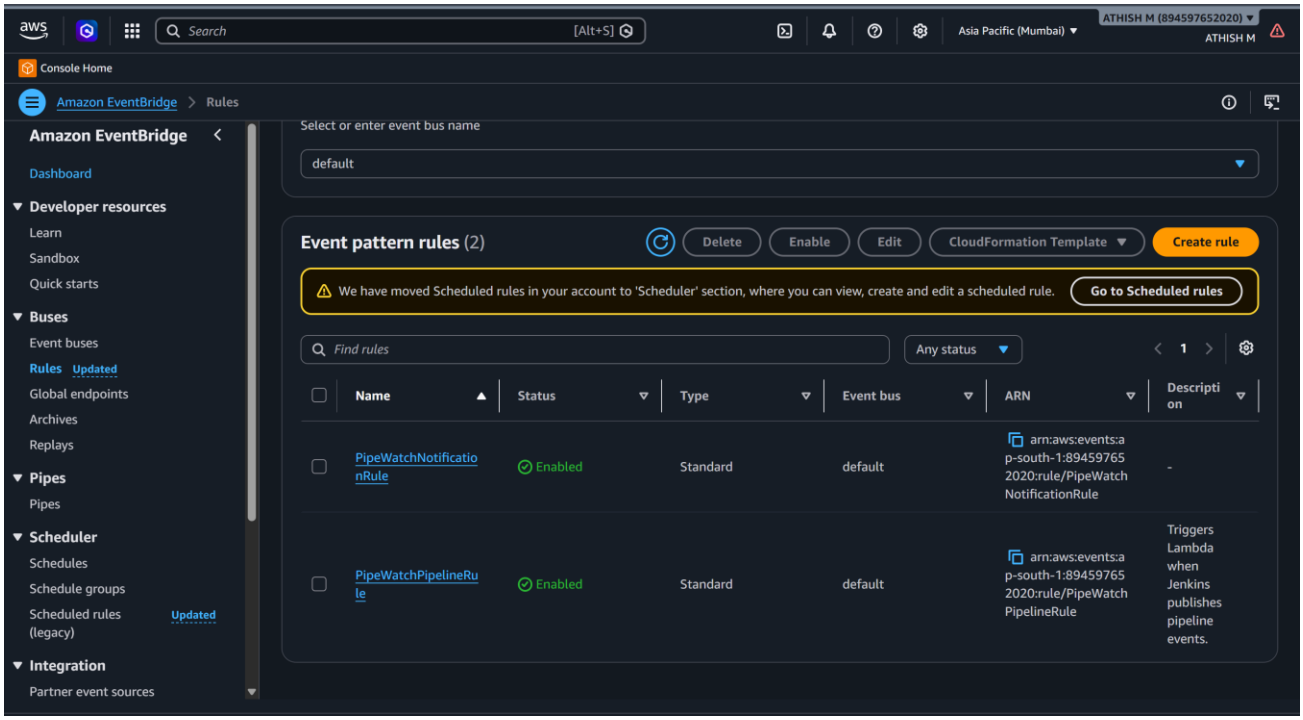
ROUTES:



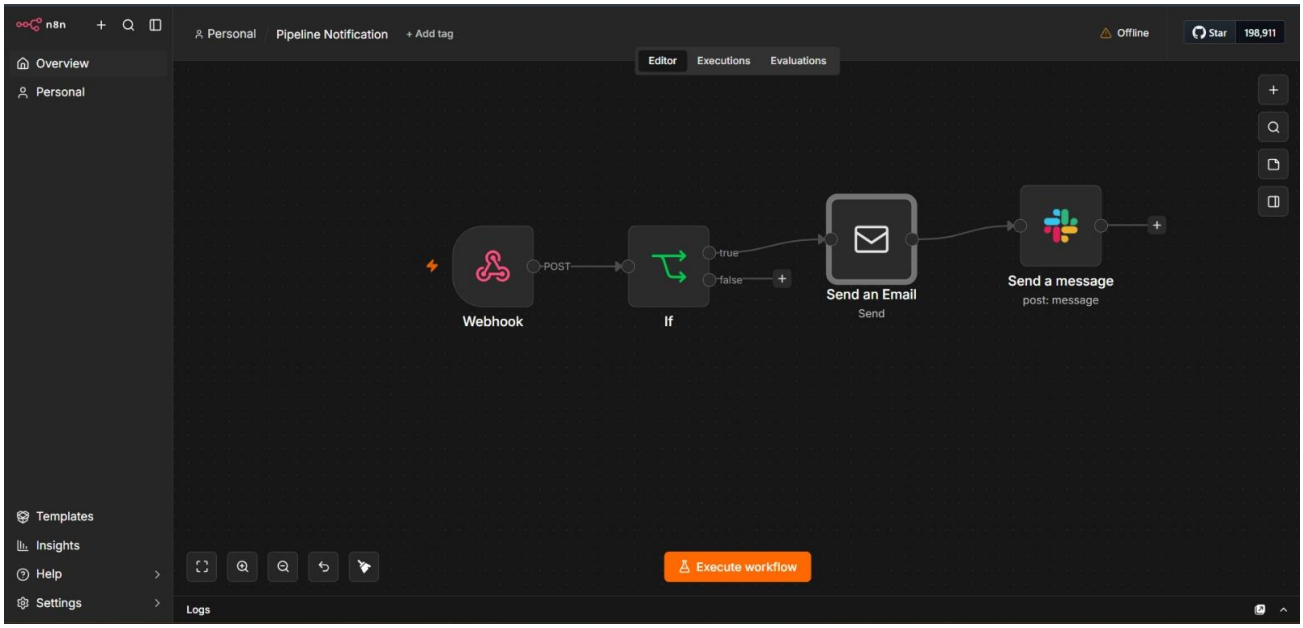
INTEGRATIONS:



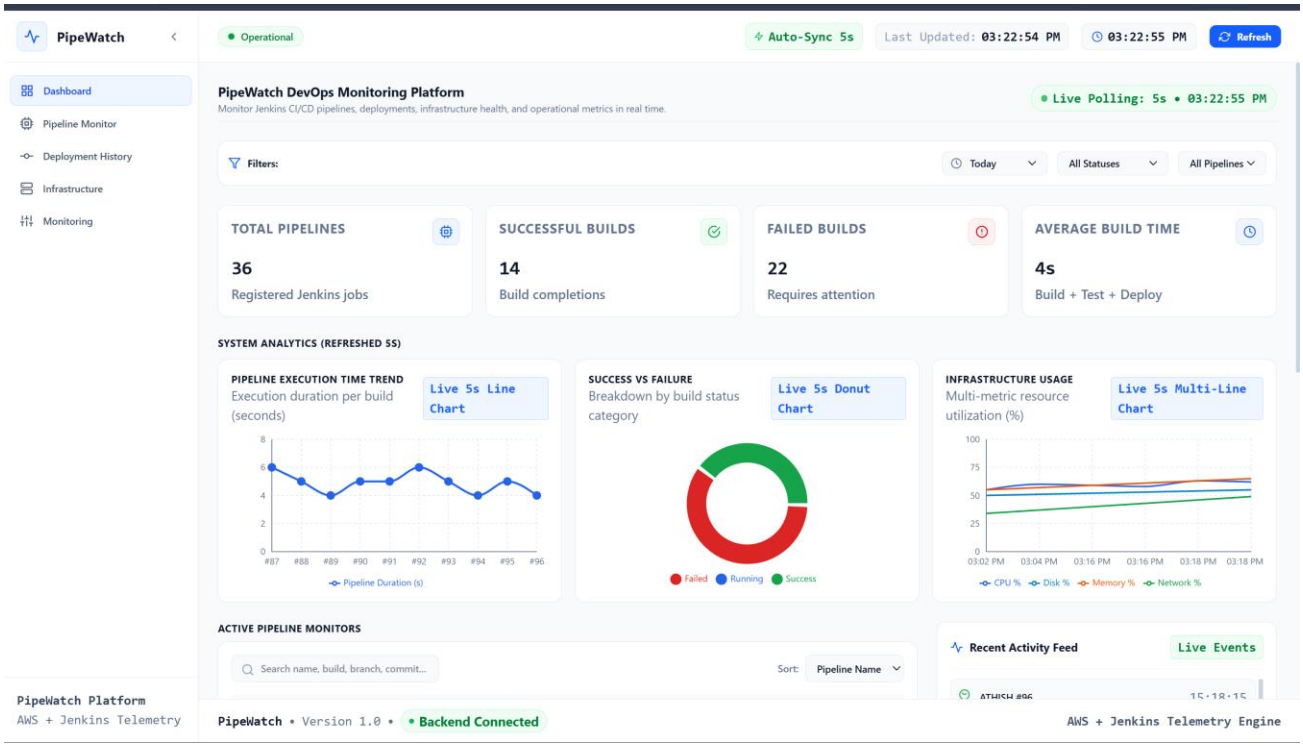
AMAZON EVENTBRIDGE:

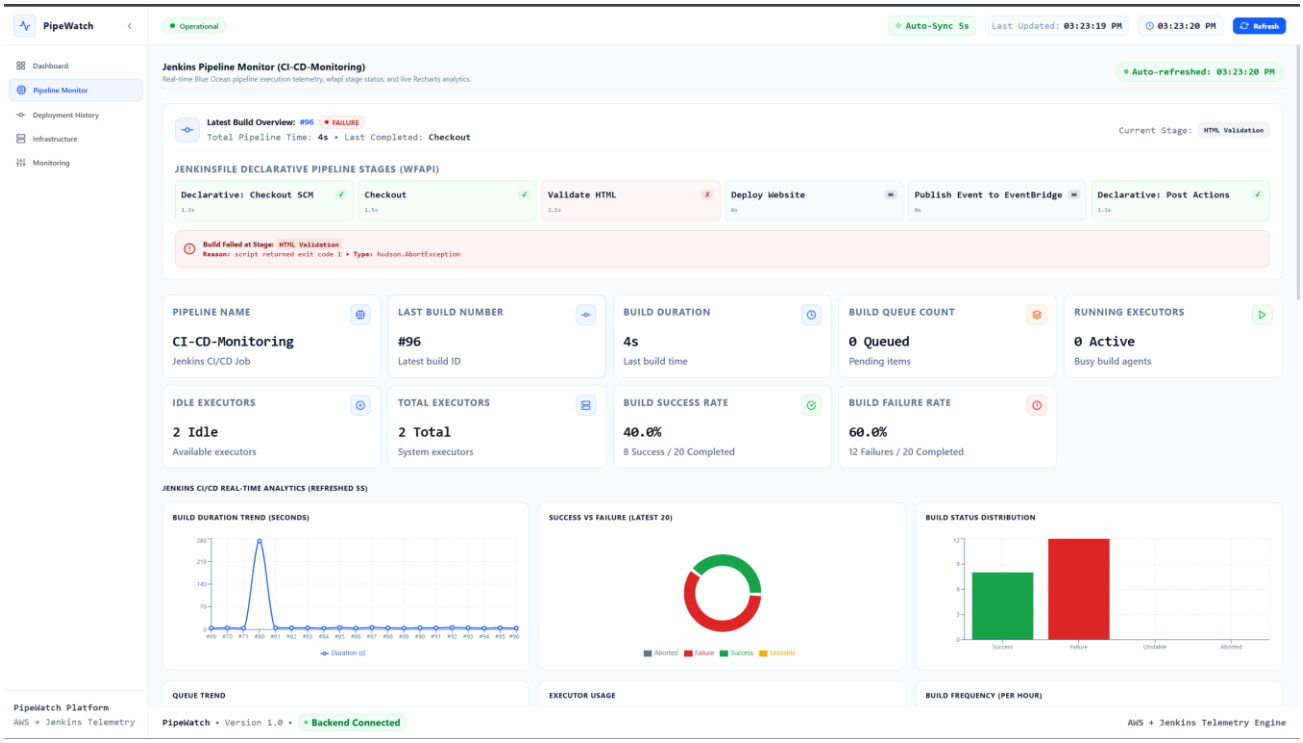


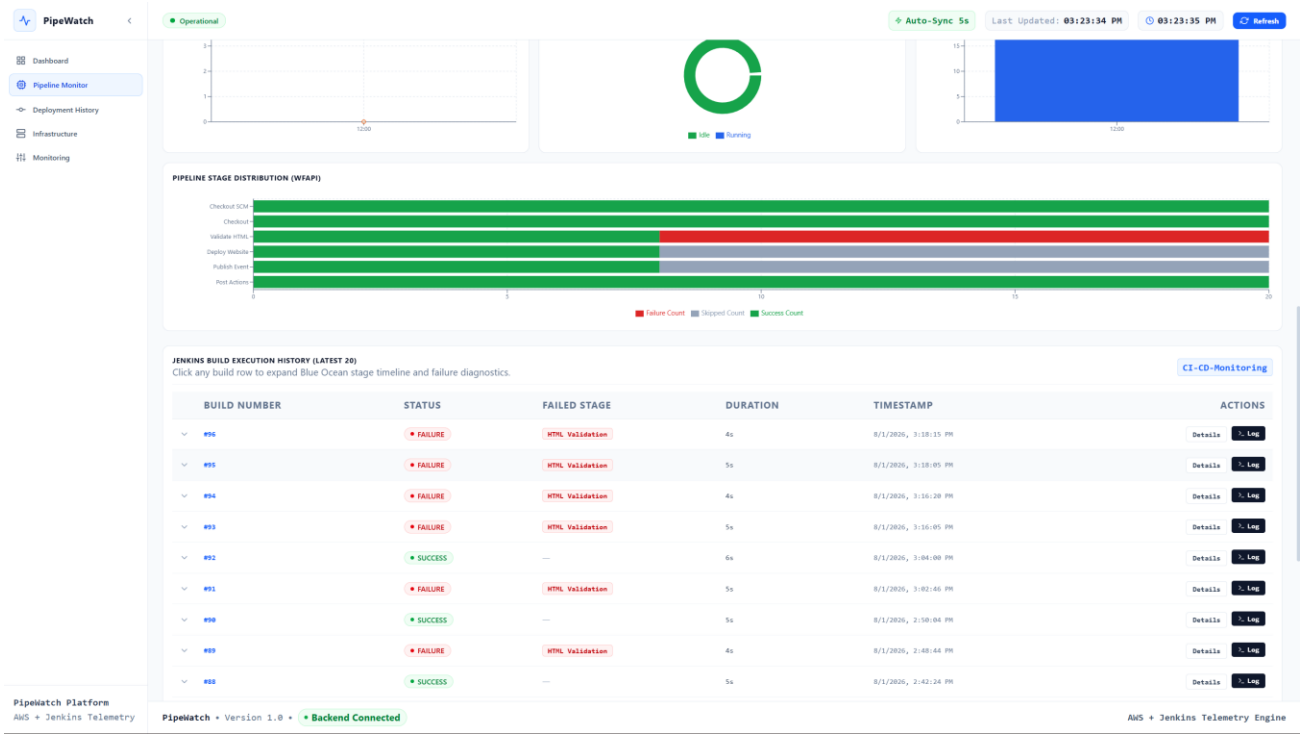
N8N:

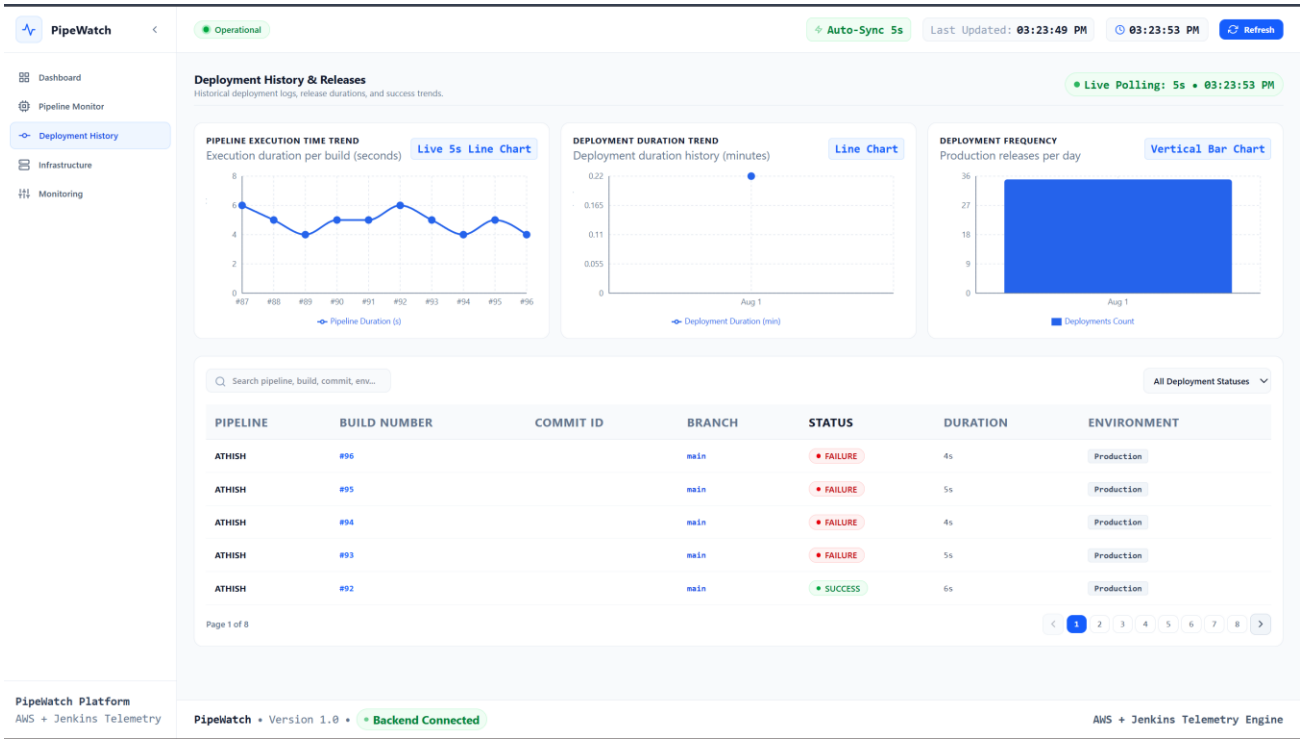


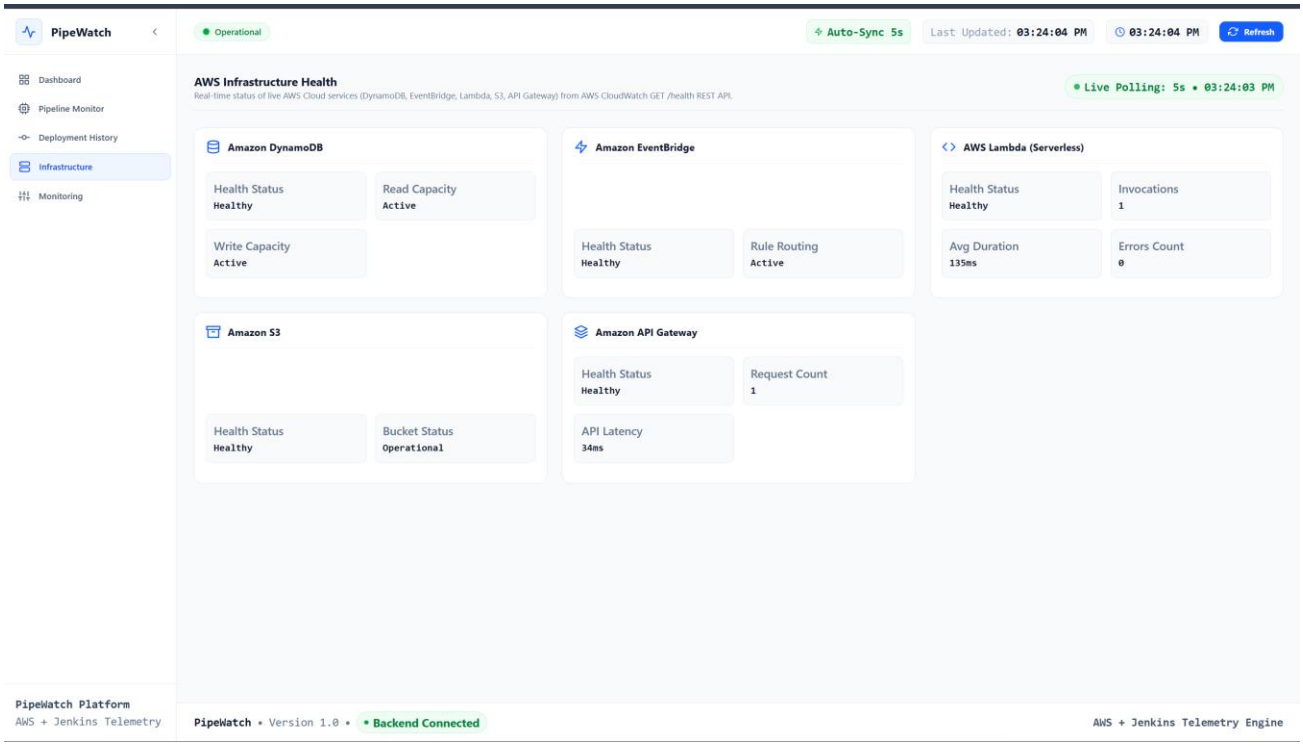
WEB-DASHBOARD:

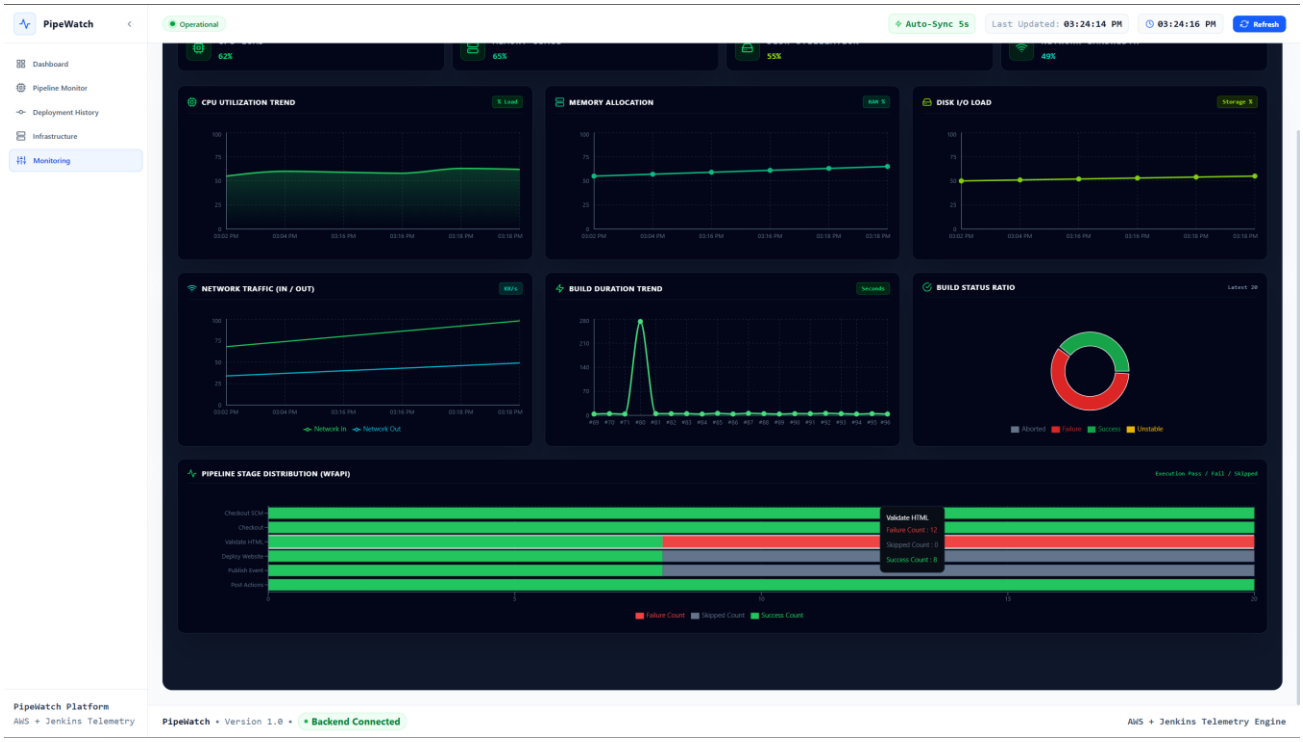





PipeWatch
Operational
Auto-Sync 5s
Last Updated: 03:23:34 PM
03:23:35 PM
Refresh
Dashboard
Pipeline Monitor
Deployment History
Infrastructure
Monitoring
PIPELINE STAGE DISTRIBUTION (WFAPI)
Checkout SCM
Checkout
Validate HTML
Deploy Website
Publish Event
Post Actions
Failure Count
Skipped Count
Success Count
JENKINS BUILD EXECUTION HISTORY (LATEST 20)
Click any build row to expand Blue Ocean stage timeline and failure diagnostics.
CI-CD-Monitoring
BUILD NUMBER
STATUS
FAILED STAGE
DURATION
TIMESTAMP
ACTIONS
#96 FAILURE HTML Validation 4s 8/1/2026, 3:18:15 PM Details Log
#95 FAILURE HTML Validation 5s 8/1/2026, 3:18:05 PM Details Log
#94 FAILURE HTML Validation 4s 8/1/2026, 3:16:20 PM Details Log
#93 FAILURE HTML Validation 5s 8/1/2026, 3:16:05 PM Details Log
#92 SUCCESS — 6s 8/1/2026, 3:04:00 PM Details Log
#91 FAILURE HTML Validation 5s 8/1/2026, 3:02:46 PM Details Log
#90 SUCCESS — 5s 8/1/2026, 2:50:04 PM Details Log
#89 FAILURE HTML Validation 4s 8/1/2026, 2:48:44 PM Details Log
#88 SUCCESS — 5s 8/1/2026, 2:42:24 PM Details Log
PipeWatch Platform
AWS + Jenkins Telemetry
PipeWatch • Version 1.0 • Backend Connected
AWS + Jenkins Telemetry Engine


PipeWatch
Operational
Auto-Sync 5s
Last Updated: 03:23:49 PM
03:23:53 PM
Refresh
Dashboard
Pipeline Monitor
Deployment History
Infrastructure
Monitoring
Deployment History & Releases
Historical deployment logs, release durations, and success trends.
Live Polling: 5s • 03:23:53 PM
PIPELINE EXECUTION TIME TREND
Execution duration per build (seconds)
Live 5s Line Chart
Pipeline Duration (s)
DEPLOYMENT DURATION TREND
Deployment duration history (minutes)
Line Chart
Deployment Duration (min)
DEPLOYMENT FREQUENCY
Production releases per day
Vertical Bar Chart
Deployments Count
Search pipeline, build, commit, env...
All Deployment Statuses
PIPELINE
BUILD NUMBER
COMMIT ID
BRANCH
STATUS
DURATION
ENVIRONMENT
ATHISH #96 main FAILURE 4s Production
ATHISH #95 main FAILURE 5s Production
ATHISH #94 main FAILURE 4s Production
ATHISH #93 main FAILURE 5s Production
ATHISH #92 main SUCCESS 6s Production
Page 1 of 8
PipeWatch Platform
AWS + Jenkins Telemetry
PipeWatch • Version 1.0 • Backend Connected
AWS + Jenkins Telemetry Engine


PipeWatch
Operational
Auto-Sync 5s
Last Updated: 03:24:04 PM
03:24:04 PM
Refresh
Dashboard
Pipeline Monitor
Deployment History
Infrastructure
Monitoring
AWS Infrastructure Health
Real-time status of live AWS Cloud services (DynamoDB, EventBridge, Lambda, S3, API Gateway) from AWS CloudWatch GET /health REST API.
Live Polling: 5s • 03:24:03 PM
Amazon DynamoDB
Health Status
Healthy
Read Capacity
Active
Write Capacity
Active
Amazon EventBridge
Health Status
Healthy
Rule Routing
Active
AWS Lambda (Serverless)
Health Status
Healthy
Invocations
1
Avg Duration
135ms
Errors Count
0
Amazon S3
Health Status
Healthy
Bucket Status
Operational
Amazon API Gateway
Health Status
Healthy
Request Count
1
API Latency
34ms
PipeWatch Platform
AWS + Jenkins Telemetry
PipeWatch • Version 1.0 • Backend Connected
AWS + Jenkins Telemetry Engine


PipeWatch
Operational
Auto-Sync 5s
Last Updated: 03:24:14 PM
03:24:16 PM
Refresh
Dashboard
Pipeline Monitor
Deployment History
Infrastructure
Monitoring
62%
65%
55%
49%
CPU UTILIZATION TREND
MEMORY ALLOCATION
DISK I/O LOAD
NETWORK TRAFFIC (IN / OUT)
Network In
Network Out
BUILD DURATION TREND
BUILD STATUS RATIO
Aborted
Failure
Success
Unstable
PIPELINE STAGE DISTRIBUTION (WFAPI)
Validate HTML
Failure Count : 12
Skipped Count : 0
Success Count : 8
Failure Count
Skipped Count
Success Count
PipeWatch Platform
AWS + Jenkins Telemetry
PipeWatch • Version 1.0 • Backend Connected
AWS + Jenkins Telemetry Engine

10.2 Performance Observations

- The dashboard reflects a new Jenkins build status within a few seconds of the pipeline completing.
- Auto-refresh keeps data current without any noticeable performance impact on the browser.
- Serverless processing through Lambda and EventBridge keeps event handling fast and cost-efficient, since resources are only used when an event actually occurs.

10.3 Advantages

- Single unified dashboard instead of switching between Jenkins, AWS Console, and logs.
- Faster detection of pipeline and deployment failures.
- Clear visual analytics on build duration, success rate, and deployment frequency.
- Event-driven architecture keeps the system responsive and cost-efficient.
- Automated notifications reduce the need for manual monitoring.

11. Future Enhancements

- Docker — containerise the backend and frontend for consistent, portable deployments.
- Kubernetes — orchestrate containers for scalability and self-healing deployments.
- Prometheus — deeper metrics collection across Jenkins and the backend.
- Grafana — richer, customizable dashboards built on top of Prometheus metrics.
- Slack — send pipeline and deployment notifications directly into team Slack channels.
- Email — automated email alerts for critical pipeline failures.
- AI Failure Prediction — use historical build data to predict likely pipeline failures before they happen.
- Multi-Pipeline Support — monitor multiple Jenkins pipelines/projects from a single dashboard.
- Role-Based Access — restrict dashboard features based on user roles (admin, developer, viewer).
- CloudWatch Agent — collect deeper EC2-level system metrics (CPU, memory, disk).
- Multi-Cloud Monitoring — extend infrastructure monitoring beyond AWS to other cloud providers.

12. Conclusion

12.1 Summary

PipeWatch successfully centralises CI/CD pipeline monitoring and AWS infrastructure monitoring into a single web-based platform. It eliminates the need to switch between multiple monitoring tools by providing live Jenkins pipeline information, deployment history, infrastructure health, and interactive analytics through one unified dashboard.

12.2 Achievements

- Built a working end-to-end CI/CD pipeline using Jenkins, GitHub, and AWS.
- Implemented an event-driven backend using EventBridge, Lambda, and DynamoDB.
- Developed a real-time, auto-refreshing React dashboard with interactive charts.
- Integrated automated notifications using n8n.

12.3 Learning Outcomes

- Hands-on experience configuring and securing core AWS services (IAM, EC2, S3, Lambda, EventBridge, API Gateway, DynamoDB).
- Practical understanding of building and troubleshooting a real Jenkins Declarative Pipeline.
- Experience building a full-stack monitoring application with Node.js/Express and React.js.
- Improved debugging skills gained from resolving real issues such as CORS errors, OOM crashes, and event routing.

12.4 Benefits

- Faster failure detection and improved deployment visibility.
- Simplified DevOps monitoring workflow for the team.
- Better operational decision-making through consolidated analytics.

Appendix

A.1 AWS Resources Created

- 1 IAM user with restricted policy and access keys
- 1 EC2 instance (Jenkins server)
- 1 S3 bucket (static website hosting)
- 1 EventBridge custom event bus and rule
- 1 Lambda function (event processing)
- 1 DynamoDB table (deployment/event records)
- 1 API Gateway REST API

A.2 Jenkins Plugins Used

- GitHub Integration Plugin
- Pipeline Plugin
- Credentials Binding Plugin
- AWS Steps / AWS CLI integration

A.3 Folder Structure (Project Root)

```
PipeWatch/
├── backend/          → Node.js + Express API server
├── frontend/         → React.js dashboard (Vite)
├── jenkins/          → Jenkinsfile and pipeline scripts
└── infra/            → AWS setup notes / IaC references
```